Planet Merge: Performance Guide

What is expensive, what we already fixed, what still lags, and what Nikola can do about it. Updated July 2026, after every planet became a plain circle collider.

TL;DR. The game has two jobs every frame: **simulate** (Matter.js physics) and **paint** (canvas drawing). Painting was solved earlier by pre-baking every SVG into a bitmap. The big news since the last revision: **every planet is now a plain circle collider**. Saturn and the Sun used to be compound bodies traced from an SVG silhouette and split by poly-decomp into dozens of convex slivers, and that was the single most expensive thing in the whole game. It is gone. What remains is ordinary cost: the number of bodies on the board times the collision solver's work (which we deliberately made a little heavier to stop planets crushing into each other), plus full-canvas paint. **There is no silhouette left to simplify**, so that Illustrator lever is retired. Simplifying the rest of the artwork now only helps download, load and bake time, not framerate.

1. The frame budget

At 60 Hz the game has **16.7 ms** per frame. In that window it must run up to 4 physics substeps (fixed 8 ms steps fed by real elapsed time) and then repaint the whole 840 x 1160 canvas. A frame that takes longer is a visible stutter. A phone has roughly a quarter of a desktop's CPU and GPU budget, which is why everything below matters ~4x more on mobile.

2. The expensive things, ranked

#1 Physics collision solving on a full board

Matter.js has no true circles: every planet is a convex polygon with **64 sides (32 on phones)**. That is now true of *all* of them. Previously Saturn and the Sun were traced from their SVG silhouette and, being concave, split into many convex sub-pieces, each its own collider, so two Suns touching meant solving dozens of polygon pairs 125 times a second. **That entire class of cost is removed** — every contact is now one convex polygon against another.

- Two deliberate additions make each step a little heavier than stock Matter, both introduced to kill the crush/spin bugs: the contact solver was stiffened to `positionIterations 14` / `velocityIterations 8` (Matter defaults are 6 / 4), and two light per-step passes run — `separatePenetrations` (an O(n^2) scan of the ~30 planet pairs that eases apart any deep overlap) and a spin limiter (an `afterUpdate` loop that damps and caps every planet's rotation).
- Cost still grows with how full the board is: 25+ bodies late game is the worst case, but a far cheaper worst case than the old compound-collider pile.
- Merge cascades are still spikes: every merge wakes all sleeping bodies (needed, see the CLAUDE.md), so during a chain nothing sleeps and the solver runs hot.
- Spawning a Saturn or Sun no longer causes the old per-spawn poly-decomp hitch. It is just a circle now, same as everything else.

#2 Painting pixels (fill rate)

Every frame repaints the container gradient, ~70 twinkling stars, every planet sprite, the ship, its beam, the big sky score, and now the decorative accessories: Saturn's ring and the Sun's corona + sunglasses. Those accessories are baked bitmaps too (blitted, never re-rasterized), and bodies now render in three layered passes — all back accessories, then all bodies + faces, then front accessories — so a ring can never cover a neighbouring planet. That is extra iteration over a ~30-item list, negligible pixels. On weak phones the canvas still renders at 0.7 scale.

- The ship beam builds 4 gradient objects per frame and uses `lighter` blending (an extra read-modify-write pass). Skipped on mobile.
- The sky score + moving ship shadow redraws an offscreen band every frame on desktop; mobile uses a cached bitmap instead.

#3 SVG rasterization (solved, but the rule must hold)

Historically the #1 lag source: browsers re-rasterize vector images on almost every `drawImage`, and the game once did it for every planet, every frame. Since the bitmap-baking rework it happens exactly once per asset at load — bodies, faces and now the accessory art all bake to bitmaps up front. **The rule: never draw an SVG-backed image inside the game loop.** Bake it, then blit. One-off UI (legend, perk tiles, start screen) may use SVGs directly.

#4 Small steady costs and dead weight

- Per-frame allocations: the live-shapes list is rebuilt every substep and every paint, the beam allocates gradients, and `separatePenetrations` walks that list. Individually tiny, together they feed the garbage collector.
- **Now unused, safe to remove:** `poly-decomp` is still loaded from a CDN and `loadOutlines()` still runs at boot, but nothing uses them — no planet has `outline: true` any more. The monolithic `planet_saturn.svg` / `planet_sun.svg` are no longer loaded by the game either (it uses the `_body`, face and accessory SVGs); the blog still links one, so keep the file but it is off the gameplay path.
- Load time: ~SVG fetches for 12 bodies, the face overlays (now including Saturn and Sun faces), ship skins and the ring/corona/sunglasses art, the Fredoka font, Matter.js + poly-decomp from CDNs, plus ~250 KB of preloaded sound. Voice-over clips (1.9 MB) load only when a perk's play button is pressed.

3. What is already in place

Problem	Shipped fix
Compound Saturn/Sun colliders were the worst case	Every planet is now a plain circle collider. Saturn's ring and the Sun's corona + sunglasses are decorative bitmaps that never collide. The whole silhouette / poly-decomp path is unused.
Small planets crushed inside big ones, and spun	Flat mass (massPower 2.0) with a hard 26x ratio cap, a stiffer solver (14 / 8 iterations), a per-step de-penetration pass, and an angular-velocity damp + cap so friction can't wind planets up.
SVG re-rasterized every frame (old worst offender)	Every SVG baked once into an offscreen bitmap; body + face pre-combined so a planet with a mood is still one drawImage.
Canvas filters force a full pixel pass	No filters in the loop. The ship's shadow on the score is a pre-baked black silhouette.
Phones are fill-rate and CPU bound	Mobile perf mode (viewport <= 700 px or coarse pointer): canvas at 0.7 scale, beam skipped, score from a cached bitmap, 32-sided colliders, autopilot hidden.
Game speed tied to refresh; fast bodies tunneling	Fixed 8 ms physics steps from an accumulator, capped at 32 ms per frame so a hitch slows the sim instead of snowballing.
Contact solver never resting	Sleeping enabled; wakes are targeted and time-boxed, then bodies settle again.
Background starfield ate the frame budget	Page starfield renders at plain resolution, ~30 fps, nebula baked once.
Audio spam during cascades	Impact sounds capped at one per kind per physics tick; pooled clips overlap without restarting.

4. What still lags, honestly

- **Body count, not shape complexity.** A full late board is now ~30 simple polygons plus the stiffer solver's extra iterations. Much lighter than the old compound pile, but still the main physics cost.
- **Cascade spikes.** A long chain wakes everything, spawns bodies, separates overlaps and fires effects in the same few frames.
- **Old / cheap phones.** Even at 0.7 scale, ~30 polygon bodies plus full-canvas paint is close to the budget; the earthquake level is the stress peak. If it bites, the cheapest knob is lowering the solver iterations on mobile.

5. What you can do (the artwork checklist)

The FPS lever moved out of the artwork. No planet uses a traced-outline collider any more, so nothing in the SVGs affects collision cost. Reworking `planet_saturn.svg` or `planet_sun.svg` does nothing for framerate now — the game does not even load them. Framerate now depends on body count and the solver, both in `physics.js`, not on any path.

Honest expectation for anchor-point cleanup: fewer anchor points on planet art improves **download size, load time and bake time**, not steady-state FPS. After load the game only ever copies bitmaps; a 500-anchor Jupiter and a 50-anchor Jupiter paint at identical speed. Still worth doing, just don't expect the late-game frame cost to change.

If you do a simplification pass, start with the heaviest gameplay-baked files:

File	Size	Note
planet_jupiter_body.svg	19.2 KB	Baked for gameplay every load
planet_moon_body.svg	10.6 KB	Baked for gameplay every load
planet_saturn.svg	15.9 KB	No longer used by the game; blog card only
planet_jupiter.svg	15.4 KB	Blog cards + legend/perk icon
planet_earth.svg	11.5 KB	Blog cards + perk icon

Illustrator / export tips: Object > Path > Simplify on busy paths, export with 1 decimal of precision, no metadata, no hidden layers, never embed raster images. Running the files through SVGO afterwards is safe as long as the `viewBox` is preserved.

Housekeeping (repo / deploy size only, zero FPS impact). Unreferenced source art (old-planets/, ship-skins/source, all-planets.svg, stray planet_moon-01.svg etc., game.js.bak) can be deleted for hygiene. On top of that, the outline collision code is now dead and removable: `loadOutlines()`, `getOutlineSets`, the silhouette machinery in physics.js, and the poly-decomp CDN script in play.html. Do not delete the root `planet_*.svg` files — the blog card grid in posts.json links them.

6. How to measure any change

- Run the site (`npm run dev`) and open `localhost:3000/games/planet-merge/play.html?dev=1`.
- Build a worst case fast: either use the new dev-panel **Scenarios** tool to save and replay a known-bad board, or set Drop > Jupiter (or Saturn / Sun), turn on Auto-Drop, and raise Speed until the board fills with heavy bodies.
- Open Chrome DevTools > Performance, set CPU throttling to 4x or 6x to imitate a phone, record ~10 seconds, and read the frame track: flat = fine, tall spiky bars = over budget. The biggest blocks are labelled

`Engine.update` (physics) or paint calls.

- There is no silhouette console line to watch any more — the old `silhouette ready for ...` benchmark is retired. Watch `Engine.update` time as the body count climbs instead.

7. Code-side ideas for later (not your job)

- Delete the dead outline path entirely: `loadOutlines`, `getOutlineSets`, `outlineUnitVerts`, and the poly-decomp CDN include. Pure removal, no behaviour change now that no planet is concave.
- Scale circle collider sides with radius: a Star does not need 64 sides, only the big planets do.
- Bake the ship beam into a per-skin bitmap (it never changes shape) to drop the 4 per-frame gradients.
- Reuse one scratch array for the live-shapes list to cut GC churn (it now also feeds `separatePenetrations`).
- If the stiffer solver ever costs too much on low-end phones, lower `positionIterations` there specifically rather than globally.

Planet Merge performance guide · regenerated from the code as of July 2026 (all-circle-collider revision) · sources: game.js, physics.js, renderer.js, config.js, tuning.js · lives at /public/games/planet-merge/performance-guide.pdf